Table of Contents

..... 1
Part 1 - Mass Properties 1
Part 2 - Geometric properties in normal operations 3
Part 3 - Initialize Simulation States 6
PART 2 SIMULINK 7
Part 3 9
function dumping ground 12


```
close all
clc
```

Part 1 - Mass Properties

DETUMBLE MODE (CUBE) mass, xyz dimensions, center of mass, inertia matrix

```
d.mass = 640; % [kg]
d.x = 2; % [m]
d.y = 2; % [m]
d.z = 2; % [m]
d.COM = [0, 0, 0];

d.Ixx = (1/12)*d.mass*(d.y^2 + d.z^2);
d.Iyy = (1/12)*d.mass*(d.x^2 + d.z^2);
d.Izz = (1/12)*d.mass*(d.x^2 + d.y^2);
d.Ixy = 0;
d.Iyz = 0;
d.Ixz = 0;

d.I = [d.Ixx, d.Ixy, d.Ixz; ...
       d.Ixy, d.Iyy, d.Iyz; ...
       d.Ixz, d.Iyz, d.Izz];

% NORMAL MODE (CUBE + SENSOR + 2X PANEL)
% mass, xyz dimensions, center of mass, inertia matrix

% bus
n.bus.mass = 500; % [kg]
n.bus.x = 2; % [m]
n.bus.y = 2; % [m]
n.bus.z = 2; % [m]
n.bus.COM = [0, 0, 0];

n.bus.Ixx = (1/12)*n.bus.mass*(n.bus.y^2 + n.bus.z^2);
n.bus.Iyy = (1/12)*n.bus.mass*(n.bus.x^2 + n.bus.z^2);
n.bus.Izz = (1/12)*n.bus.mass*(n.bus.x^2 + n.bus.y^2);
n.bus.Ixy = 0;
n.bus.Iyz = 0;
n.bus.Ixz = 0;
```

```
n.bus.I = [n.bus.Ixx, n.bus.Ixy, n.bus.Ixz; ...
           n.bus.Ixy, n.bus.Iyy, n.bus.Iyz; ...
           n.bus.Ixz, n.bus.Iyz, n.bus.Izz];

% sensor
n.sens.mass = 100; % [kg]
n.sens.x = 0.25; % [m]
n.sens.y = 0.25; % [m]
n.sens.z = 1; % [m]
n.sens.COM = [0, 0, 1.5];

n.sens.Ixx = (1/12)*n.sens.mass*(n.sens.y^2 + n.sens.z^2);
n.sens.Iyy = (1/12)*n.sens.mass*(n.sens.x^2 + n.sens.z^2);
n.sens.Izz = (1/12)*n.sens.mass*(n.sens.x^2 + n.sens.y^2);
n.sens.Ixy = 0;
n.sens.Iyz = 0;
n.sens.Ixz = 0;
n.sens.I = [n.sens.Ixx, n.sens.Ixy, n.sens.Ixz; ...
            n.sens.Ixy, n.sens.Iyy, n.sens.Iyz; ...
            n.sens.Ixz, n.sens.Iyz, n.sens.Izz];

% left panel (-y)
n.panL.mass = 20; % [kg]
n.panL.x = 2; % [m]
n.panL.y = 3; % [m]
n.panL.z = 0.05; % [m]
n.panL.COM = [0, -2.5, 0];

n.panL.Ixx = (1/12)*n.panL.mass*(n.panL.y^2 + n.panL.z^2);
n.panL.Iyy = (1/12)*n.panL.mass*(n.panL.x^2 + n.panL.z^2);
n.panL.Izz = (1/12)*n.panL.mass*(n.panL.x^2 + n.panL.y^2);
n.panL.Ixy = 0;
n.panL.Iyz = 0;
n.panL.Ixz = 0;
n.panL.I = [n.panL.Ixx, n.panL.Ixy, n.panL.Ixz; ...
            n.panL.Ixy, n.panL.Iyy, n.panL.Iyz; ...
            n.panL.Ixz, n.panL.Iyz, n.panL.Izz];

% right panel (+y)
n.panR.mass = 20; % [kg]
n.panR.x = 2; % [m]
n.panR.y = 3; % [m]
n.panR.z = 0.05; % [m]
n.panR.COM = [0, 2.5, 0];

n.panR.Ixx = (1/12)*n.panR.mass*(n.panR.y^2 + n.panR.z^2);
n.panR.Iyy = (1/12)*n.panR.mass*(n.panR.x^2 + n.panR.z^2);
n.panR.Izz = (1/12)*n.panR.mass*(n.panR.x^2 + n.panR.y^2);
n.panR.Ixy = 0;
n.panR.Iyz = 0;
n.panR.Ixz = 0;
n.panR.I = [n.panR.Ixx, n.panR.Ixy, n.panR.Ixz; ...
            n.panR.Ixy, n.panR.Iyy, n.panR.Iyz; ...
            n.panR.Ixz, n.panR.Iyz, n.panR.Izz];
```

```
% total satellite
n.sat.mass = n.bus.mass + n.sens.mass + n.panL.mass + n.panR.mass; % [kg]
n.sat.COM = ((n.bus.mass*n.bus.COM) + (n.sens.mass*n.sens.COM) +
(n.panL.mass*n.panL.COM) + (n.panR.mass*n.panR.COM))/n.sat.mass;

% sum all inertia matrices with some parallel axis theorem
n.sat.I = (n.bus.I  - n.bus.mass *(vect2cross(n.sat.COM - n.bus.COM))
*(vect2cross(n.sat.COM - n.bus.COM))) + ...
          (n.sens.I - n.sens.mass*(vect2cross(n.sat.COM -
n.sens.COM))*(vect2cross(n.sat.COM - n.sens.COM))) + ...
          (n.panL.I - n.panL.mass*(vect2cross(n.sat.COM -
n.panL.COM))*(vect2cross(n.sat.COM - n.panL.COM))) + ...
          (n.panR.I - n.panR.mass*(vect2cross(n.sat.COM -
n.panR.COM))*(vect2cross(n.sat.COM - n.panR.COM)));

% RESPONSES TO FINAL PROJECT PART 1
fprintf("PART 1\n")
fprintf("Detumble total mass = %.0f kg\n", d.mass)
fprintf("Detumble COM =\n")
disp(d.COM)
fprintf("Detumble inertia matrix =\n")
disp(d.I)

fprintf("Normal total mass = %.0f kg\n", n.sat.mass)
fprintf("Normal COM =\n")
disp(n.sat.COM)
fprintf("Normal inertia matrix =\n")
disp(n.sat.I)
```

Part 2 - Geometric properties in normal operations

Define the sun in F_ECI and residual dipole moment in F_b

```
sun_ECI = [1; 0; 0];
m_b = [0; 0; -0.5];

% I constructed a matrix where the rows represent each surface of the
% MehielSat.  The first column stores the Aera of the surface, the next
% three columns define the normal vector of that surface in F_b, and the
% final three columns store the center of presure of the surface (geometric
% center of the surface) in F_b.

% First get rhos vectors with respect to the center of the spacecraft bus
% the MehielSat BUS is a box
% order +x, -x, +x, -y, +z, -z
% n.bus.areas = 4*ones(6,1);

n.bus.areas = [n.bus.y*n.bus.z;
               n.bus.y*n.bus.z;
               n.bus.x*n.bus.z - n.panL.x*n.panL.z;
```

```
                n.bus.x*n.bus.z - n.panL.x*n.panL.z;
                n.bus.x*n.bus.y - n.sens.x*n.sens.y;
                n.bus.x*n.bus.y];

n.bus.normal = [1 0 0;
                -1 0 0;
                 0 1 0;
                 0 -1 0;
                 0 0 1;
                 0 0 -1];

n.bus.cp = [1 0 0;
           -1 0 0;
            0 1 0;
            0 -1 0;
            0 0 1;
            0 0 -1];

% Append geometric properties for Solar Panel L
% order +x, -x, NA, -y, +z, -z
n.panL.areas = [n.panL.y*n.panL.z;
                n.panL.y*n.panL.z;

                n.panL.x*n.panL.z;
                n.panL.x*n.panL.y;
                n.panL.x*n.panL.y]; % [m^2]

n.panL.normal = [1 0 0;
                 -1 0 0;

                  0 -1 0;
                  0 0 1;
                  0 0 -1];

n.panL.cp = [n.panL.x/2, -(n.bus.y/2)-n.panL.y/2, 0;
            -n.panL.x/2, -(n.bus.y/2)-n.panL.y/2, 0;

             0, -(n.bus.y/2)-n.panL.y, 0;
             0, -(n.bus.y/2)-n.panL.y/2, n.panL.z/2;
             0, -(n.bus.y/2)-n.panL.y/2, -n.panL.z/2];

% Append geometric properties for Solar Panel R
% order +x, -x, +y, NA, +z, -z
n.panR.areas = [n.panR.y*n.panR.z;
                n.panR.y*n.panR.z;

                n.panR.x*n.panR.z;
                n.panR.x*n.panR.y;
                n.panR.x*n.panR.y]; % [m^2]

n.panR.normal = [1 0 0;
                 -1 0 0;
                  0 1 0;
```

```
                 0 0 1;
                 0 0 -1];

n.panR.cp = [n.panR.x/2, (n.bus.y/2)+n.panR.y/2, 0;
            -n.panR.x/2, (n.bus.y/2)+n.panR.y/2, 0;
             0, (n.bus.y/2)+n.panR.y, 0;

             0, (n.bus.y/2)+n.panR.y/2, n.panR.z/2;
             0, (n.bus.y/2)+n.panR.y/2, -n.panR.z/2];

% Append geometric properties for Sensor
% order +x, -x, +y, -y, +z, NA
n.sens.areas = [n.sens.y*n.sens.z;
                n.sens.y*n.sens.z;
                n.sens.x*n.sens.z;
                n.sens.x*n.sens.z;
                n.sens.x*n.sens.y;

                ]; % [m^2]

n.sens.normal = [1 0 0;
                -1 0 0;
                 0 1 0;
                 0 -1 0;
                 0 0 1;

                ];

n.sens.cp = [n.sens.x/2, 0, (n.bus.z/2)+n.sens.z/2;
            -n.sens.x/2, 0, (n.bus.z/2)+n.sens.z/2;
             0, n.sens.y/2, (n.bus.z/2)+n.sens.z/2;
             0, -n.sens.y/2, (n.bus.z/2)+n.sens.z/2;
             0, 0, (n.bus.z/2)+n.sens.z

             ];

% now subtract the center of mass to get the location of the rho vectors
% with respect to the center of mass
n.sat.areas = [n.bus.areas;
               n.panL.areas;
               n.panR.areas;
               n.sens.areas]; % [m^2]

n.sat.normal = [n.bus.normal;
                n.panL.normal;
                n.panR.normal;
                n.sens.normal];

n.sat.cp = [n.bus.cp;
            n.panL.cp;
            n.panR.cp;
            n.sens.cp];

n.sat.rho = n.sat.cp - n.sat.COM;
```

```
% Now build the matrix
% surfaceProperties = [n.sat.areas n.sat.cp n.sat.normal];
surfaceProperties = [n.sat.areas n.sat.rho n.sat.normal];
```

Part 3 - Initialize Simulation States

```
% Current JD - has to be on the solar equinox, why? - we'll use 3/20/2024
% from https://aa.usno.navy.mil/data/JulianDate
% Need this so we can convert from F_ECEF to F_ECI and to F_b for the
% magnetic field model
JD_0 = 2460390;

% Spacecraft Orbit Properties
mu = 398600; % km^3/s^2
h = 53335.2; % km^2/s
ecc = 0; % none
raan = deg2rad(0); % [rad]
inc = deg2rad(98.43); % [rad]
argp = deg2rad(0); % [rad]
ta = deg2rad(0); % [rad]

a = h^2/mu/(1 - ecc^2);
orbital_period = 2*pi*sqrt(a^3/mu);

% Set/Compute initial conditions
% intial orbital position and velocity
% % [r_ECI_0, v_ECI_0] = coe2rv([h ecc Omega inc omega nu]);
[r0eci_vec, v0eci_vec] = coes2rv(h, inc, raan, ecc, argp, ta, mu);

% No external command Torque
T_c = [0; 0; 0]; % Nm

% Compute inital F_LVLH basis vectors in F_ECI components based on F_LVLH
% definition

% Initial Euler angles relating F_body and F_LVLH (given)
phi0 = 0;
theta0 = 0;
psi0 = 0;
EA0_b_lvlh = [phi0; theta0; psi0];

% Initial Quaternion relating F_body and F_LVLH (given)
q0_b_lvlh = [0; 0; 0; 1];
EAq0_b_lvlh = [EA0_b_lvlh; q0_b_lvlh];

% Compute initial C_LVLH_ECI_0, C_b_LHVL_0, and C_b_ECI_0 rotaiton matrices
C_lvlh_eci = ECItoLVLH(r0eci_vec, v0eci_vec);
C_b_lvlh = build_eulerDCM(3, 2, 1, phi0, theta0, psi0, false);
C_b_eci = C_b_lvlh * C_lvlh_eci;

% Initial Euler angles relating body to ECI
% E_b_ECI_0 = C2EulerAngles(C_b_ECI_0);
```

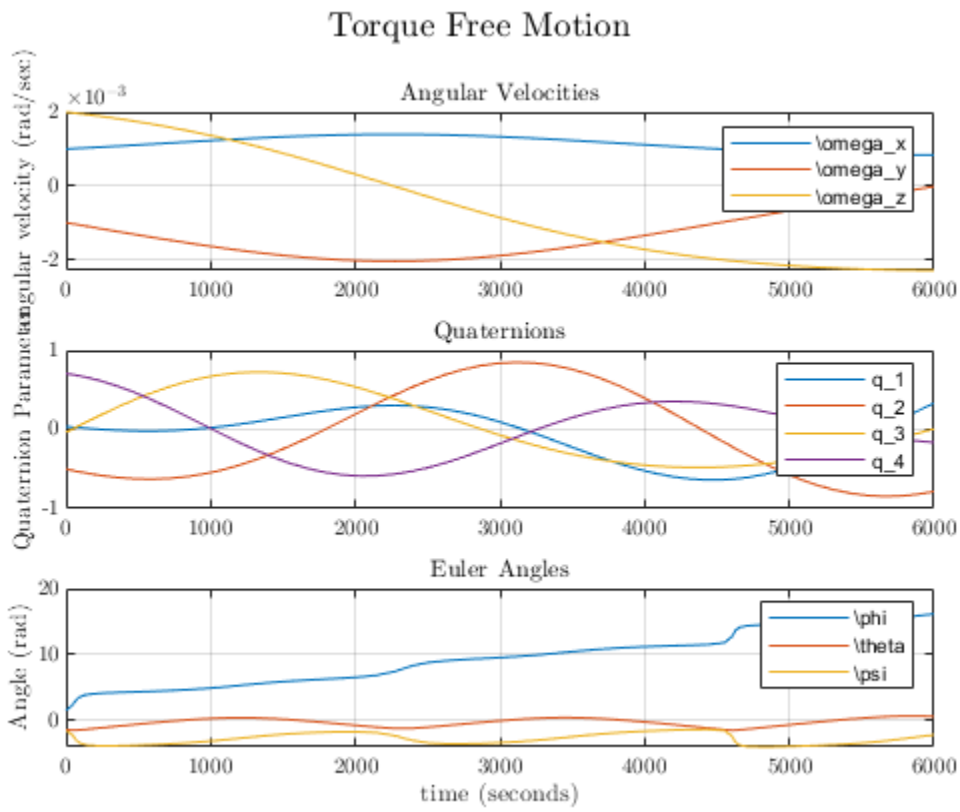
```
[phi_b_eci_0, theta_b_eci_0, psi_b_eci_0] = find_321_angles(C_b_eci, false);
Eul0_b_eci = [phi_b_eci_0; theta_b_eci_0; psi_b_eci_0];

% Initial quaternion relating body to E
% q_b_ECI_0 = C2quat(C_b_ECI_0);
q0_b_eci = DCM_to_quat(C_b_eci);
q0_b_eci = q0_b_eci(:);  % force 4x1 column vector

% Initial body rates of spacecraft (given)
w_b_ECI_0 = [0.001; -0.001; 0.002];
% multiple naming conventions...
w0_b_eci = w_b_ECI_0;
```

PART 2 SIMULINK

```
% Set simulation time period
tspan = orbital_period;

% Simulate!
T = T_c;
J_sc = n.sat.I;

% make sure everything is a column bc matlab angy
w_b_ECI_0  = w_b_ECI_0(:);
q0_b_eci   = q0_b_eci(:);
Eul0_b_eci = Eul0_b_eci(:);

disp(T)
disp(J_sc)
out_noT = sim('ORR_Part2.slx');

out_noT

% Plot Results
figure(1)
subplot(3,1,1)
plot(out_noT.tout, squeeze(out_noT.w.signals.values))
% plot(out.w_b_ECI(:,1), out.w_b_ECI(:,2:4))
title('Angular Velocities')
ylabel('angular velocity (rad/sec)')
legend('\omega_x','\omega_y','\omega_z')
grid on

subplot(3,1,2)
plot(out_noT.tout, squeeze(out_noT.q.signals.values))
% plot(out.tout, out.q_b_ECI(:,2:5))
title('Quaternions')
ylabel('Quaternion Parameter')
legend('q_1','q_2','q_3','q_4')
grid on

subplot(3,1,3)
plot(out_noT.tout, squeeze(out_noT.E.signals.values))
```

```
% plot(out.tout, out.E_b_ECI(:,2:4))
title('Euler Angles')
xlabel('time (seconds)')
ylabel('Angle (rad)')
legend('\phi','\theta','\psi')
grid on

sgtitle('Torque Free Motion')
```

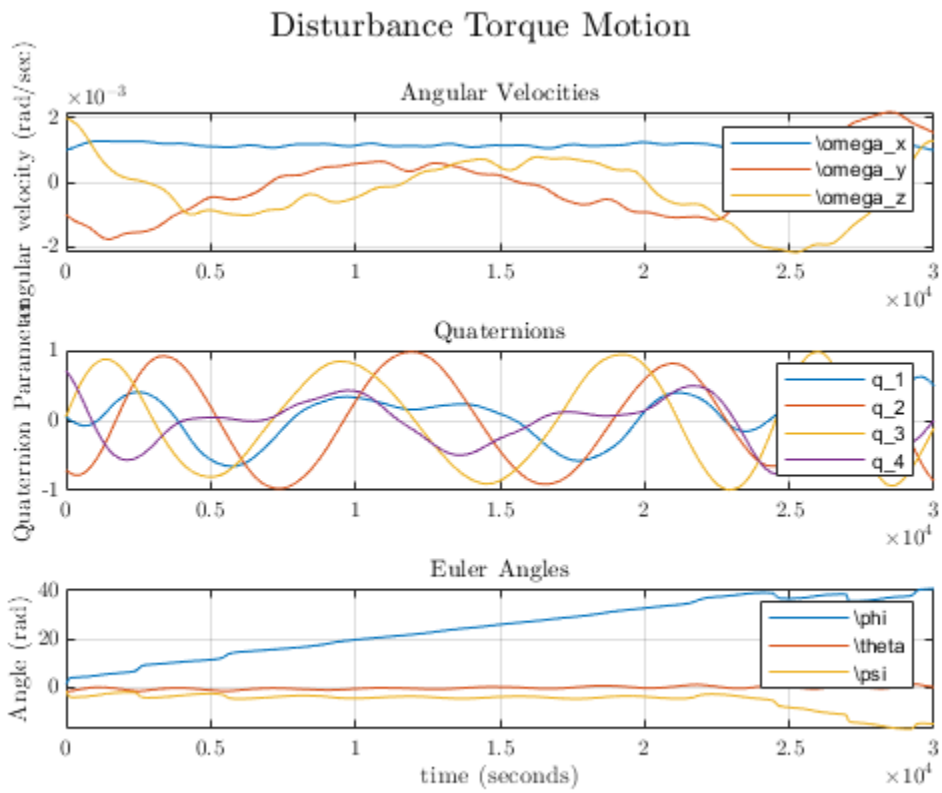
```
     0
     0
     0

  812.0396         0         0
         0  545.3729         0
         0         0  627.7083


out_noT =

  Simulink.SimulationOutput:
                      E: [1x1 struct]
                      q: [1x1 struct]
                   tout: [58x1 double]
                      w: [1x1 struct]

     SimulationMetadata: [1x1 Simulink.SimulationMetadata]
           ErrorMessage: [0x0 char]
```

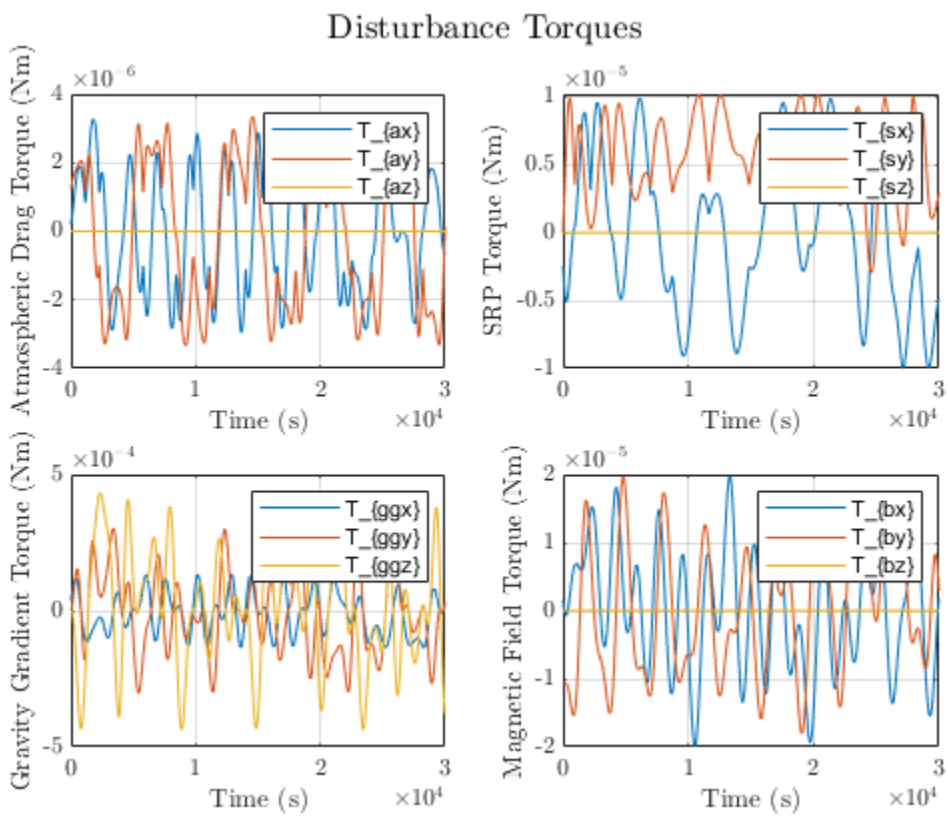


Part 3

```
n_revs = 5; %revs
tspan = n_revs * orbital_period;

% fix some syntax before passing into simulink
r_ECI_0 = r0eci_vec;  % [km]
v_ECI_0 = v0eci_vec;  % [km/s]
J_sc = n.sat.I;  % plain matrix

out_T = sim('Part3');

% Plot Results

% Plot Angular Velocities, Euler Angles and Quaternions
figure(2)
subplot(3,1,1)
plot(out_T.tout, squeeze(out_T.w_b_ECI.signals.values))
title('Angular Velocities')
ylabel('angular velocity (rad/sec)')
legend('\omega_x','\omega_y','\omega_z')
grid on

subplot(3,1,2)
plot(out_T.tout, squeeze(out_T.q_b_ECI.signals.values))
```

```
title('Quaternions')
ylabel('Quaternion Parameter')
legend('q_1','q_2','q_3','q_4')
grid on

subplot(3,1,3)
plot(out_T.tout, squeeze(out_T.E_b_ECI.signals.values))
title('Euler Angles')
xlabel('time (seconds)')
ylabel('Angle (rad)')
legend('\phi','\theta','\psi')
grid on

sgtitle('Disturbance Torque Motion')

% Plot Disturbance torques in F_b
figure(3)
subplot(2,2,1)
plot(out_T.tout, squeeze(out_T.T_a.signals.values))
xlabel('Time (s)')
ylabel('Atmospheric Drag Torque (Nm)')
legend('T_{ax}','T_{ay}','T_{az}')
grid on

subplot(2,2,2)
plot(out_T.tout, squeeze(out_T.T_srp.signals.values))
xlabel('Time (s)')
ylabel('SRP Torque (Nm)')
legend('T_{sx}','T_{sy}','T_{sz}')
grid on

subplot(2,2,3)
plot(out_T.tout, squeeze(out_T.T_gg.signals.values))
xlabel('Time (s)')
ylabel('Gravity Gradient Torque (Nm)')
legend('T_{ggx}','T_{ggy}','T_{ggz}')
grid on

subplot(2,2,4)
plot(out_T.tout, squeeze(out_T.T_b.signals.values))
xlabel('Time (s)')
ylabel('Magnetic Field Torque (Nm)')
legend('T_{bx}','T_{by}','T_{bz}')
grid on

sgtitle('Disturbance Torques')
```


Disturbance Torque Motion
Angular Velocities
Angular velocity (rad/sec)
\omega_x
\omega_y
\omega_z
Quaternions
Quaternion Parameters
q_1
q_2
q_3
q_4
Euler Angles
Angle (rad)
\phi
\theta
\psi
time (seconds)


Disturbance Torques
Atmospheric Drag Torque (Nm)
T_{ax}
T_{ay}
T_{az}
SRP Torque (Nm)
T_{sx}
T_{sy}
T_{sz}
Gravity Gradient Torque (Nm)
T_{ggx}
T_{ggy}
T_{ggz}
Magnetic Field Torque (Nm)
T_{bx}
T_{by}
T_{bz}
Time (s)

function dumping ground

```
function vect2cross = vect2cross(v)
    vect2cross = [ 0,    -v(3),  v(2)
              v(3),    0,   -v(1)
             -v(2),   v(1),   0];
end


function [r_vec, v_vec] = coes2rv(h, inc, raan, ecc, argp, ta, mu)
% convert 6 classical orbital elements to r and v vector
    % INPUT:
        %  h - angular momentum [km^2/s]
        %  inc - inclination [rad]
        %  raan - right ascension of ascending node [rad]
        %  ecc - eccentricity
        %  argp - argument of perigee [rad]
        %  ta - true anomaly [rad]
        %  mu - gravitational parameter [km^3/s^2]
    % OUTPUT:
        %  rvec - position vector [km]
        %  vvec - velocity vector [km/s]

% position vector in perifocal frame
rPQWvec = (h^2/mu)*(1/(1+ecc*cos(ta)))*([cos(ta); sin(ta); 0]); % [km]

% velocity vector in perifocal frame
vPQWvec = (mu/h)*([-sin(ta); ecc+cos(ta); 0]); % [km/s]

% ECI to perifocal rotation
R3_raan = [ cos(raan),  sin(raan), 0;
           -sin(raan),  cos(raan), 0;
                    0,          0, 1];

R1_inc = [1,          0,         0;
            0, cos(inc),  sin(inc);
            0,-sin(inc),  cos(inc)];

R3_argp = [ cos(argp),  sin(argp), 0;
           -sin(argp),  cos(argp), 0;
                    0,          0, 1];

R_EtoP = R3_argp * R1_inc * R3_raan; % [km]

% perifocal to ECI
R_PtoE = R_EtoP.'; % [km]

% convert r and v vectors from PQW to ECI frame
r_vec = R_PtoE * rPQWvec; % [km] ECI
v_vec = R_PtoE * vPQWvec; % [km/s] ECI

end
```

```
function [C_LVLH_ECI] = ECItoLVLH(r_eci, v_eci)
% getLVLHFrame
% Build LVLH basis vectors and the DCM relating ECI to LVLH

    z_lvlh_eci = -r_eci / norm(r_eci);
    y_lvlh_eci = -cross(r_eci, v_eci) / norm(cross(r_eci, v_eci));
    x_lvlh_eci = cross(y_lvlh_eci, z_lvlh_eci);

    C_LVLH_ECI = [x_lvlh_eci.';
                  y_lvlh_eci.';
                  z_lvlh_eci.'];
end


function DCM = build_eulerDCM(first_rot, second_rot, third_rot, a1, a2, a3,
degs)
% build_eulerDCM
% Builds direction cosine matrix for any 3-axis rotation sequence
% Based off fundamentals of d&c, ch. 1.3.3
%
% Note:
%   1 = x-axis
%   2 = y-axis
%   3 = z-axis
%
% You must enter the rotation angles in the order they occur.
%
% Inputs:
%   first_rot  : first rotation axis number
%   second_rot : second rotation axis number
%   third_rot  : third rotation axis number
%   a1         : first rotation angle
%   a2         : second rotation angle
%   a3         : third rotation angle
%   degs       : true if input angles are in degrees
%
% Output:
%   DCM        : 3x3 direction cosine matrix

    if nargin < 7
        degs = false;
    end

    if nargin < 6
        a3 = [];
    end
    if nargin < 5
        a2 = [];
    end
    if nargin < 4
        a1 = [];
    end
```

```
    if isempty(a1) && isempty(a2) && isempty(a3)
        symbolic = true;

        angle_names = {'phi', 'theta', 'psi'};
        a1 = sym(angle_names{first_rot}, 'real');
        a2 = sym(angle_names{second_rot}, 'real');
        a3 = sym(angle_names{third_rot}, 'real');
    else
        symbolic = false;
    end

    if degs && ~symbolic
        a1 = deg2rad(a1);
        a2 = deg2rad(a2);
        a3 = deg2rad(a3);
    end

    switch first_rot
        case 1
            matrix1 = Cx(a1, symbolic);
        case 2
            matrix1 = Cy(a1, symbolic);
        case 3
            matrix1 = Cz(a1, symbolic);
        otherwise
            error('first_rot must be 1, 2, or 3');
    end

    switch second_rot
        case 1
            matrix2 = Cx(a2, symbolic);
        case 2
            matrix2 = Cy(a2, symbolic);
        case 3
            matrix2 = Cz(a2, symbolic);
        otherwise
            error('second_rot must be 1, 2, or 3');
    end

    switch third_rot
        case 1
            matrix3 = Cx(a3, symbolic);
        case 2
            matrix3 = Cy(a3, symbolic);
        case 3
            matrix3 = Cz(a3, symbolic);
        otherwise
            error('third_rot must be 1, 2, or 3');
    end

    DCM = matrix3 * matrix2 * matrix1;
end

% rotation functions Cx, Cy, Cz
```

```
function C = Cx(a, ~)
    c = cos(a); s = sin(a);
    C = [1, 0, 0; 0, c, s; 0,-s, c];
end

function C = Cy(a, ~)
    c = cos(a); s = sin(a);
    C = [c, 0,-s; 0, 1, 0; s, 0, c];
end

function C = Cz(a, ~)
    c = cos(a); s = sin(a);
    C = [ c, s, 0; -s, c, 0; 0, 0, 1];
end


function quat = DCM_to_quat(C)
% DCM_to_quat
% DCM to quaternion
% Returns q = [e1; e2; e3; eta], normalized.
%
% Supports numeric or symbolic input.

    tol = 1e-8;

    if isa(C, 'sym')
        tr = trace(C);
        eta = sqrt(tr + 1)/2;

        if abs(double(vpa(eta))) > tol
            e1 = (C(3,2) - C(2,3)) / (4*eta);
            e2 = (C(1,3) - C(3,1)) / (4*eta);
            e3 = (C(2,1) - C(1,2)) / (4*eta);
        else
            e1 = sqrt((C(1,1) + 1)/2);
            e2 = sqrt((C(2,2) + 1)/2);
            e3 = sqrt((C(3,3) + 1)/2);

            % arbitrarily choose e1 positive
            e1 = abs(e1);
            e2 = sign(C(1,2)) * abs(e2);
            e3 = sign(C(1,3)) * abs(e3);
        end

        quat = [e1; e2; e3; eta];
        quat = quat / norm(quat);

    else
        C = reshape(double(C), 3, 3);
        tr = trace(C);
        eta = 0.5 * sqrt(tr + 1.0);

        if abs(eta) > tol
            e1 = (C(2,3) - C(3,2)) / (4.0*eta);
```

```
            e2 = (C(3,1) - C(1,3)) / (4.0*eta);
            e3 = (C(2,1) - C(1,2)) / (4.0*eta);
        else
            e1 = sqrt((C(1,1) + 1.0)/2.0);
            e2 = sqrt((C(2,2) + 1.0)/2.0);
            e3 = sqrt((C(3,3) + 1.0)/2.0);

            % arbitrarily choose e1 positive
            e1 = abs(e1);
            e2 = sign(C(1,2)) * abs(e2);
            e3 = sign(C(1,3)) * abs(e3);
        end

        quat = [e1; e2; e3; eta];
        quat = quat / norm(quat);
    end
end


function [phi, theta, psi] = find_321_angles(C, degs)
% find_321_angles
% Extract 3-2-1 Euler angles from a DCM

    if nargin < 2
        degs = false;
    end

    phi = atan2(C(2,3), C(3,3));
    theta = -asin(C(1,3));
    psi = atan2(C(1,2), C(1,1));

    if degs
        phi = rad2deg(phi);
        theta = rad2deg(theta);
        psi = rad2deg(psi);
    end
end
```

```
PART 1
Detumble total mass = 640 kg
Detumble COM =
     0     0     0

Detumble inertia matrix =
  426.6667         0         0
         0  426.6667         0
         0         0  426.6667

Normal total mass = 640 kg
Normal COM =
         0         0    0.2344

Normal inertia matrix =
  812.0396         0         0
```

```
         0  545.3729         0
         0         0  627.7083
```

Published with MATLAB® R2024b